

Section 1.1: Scale Engineering Primitives & Performance Matrices

Enhanced Edition — Structured for study flow with explicit cause→approach analysis

Executive Overview

Scale engineering is the discipline of designing systems that maintain predictable behavior under increasing load. While performance optimization improves response times on a single node, scalability engineering expands capacity horizontally. This distinction matters in interviews: interviewers probe whether candidates can architect beyond the single-machine bottleneck. The core primitives—performance vs. scalability, latency vs. throughput, concurrency models—form the foundation of every architectural decision.

How to use this section: Each topic follows the structure of *What It Is* → *Dimensions (with Cause→Approach)* → *Cross-Connections* → *Failure Modes* → *Interview Questions*. The interview questions are graded: ★ = mid-level, ★★ = senior, ★★★ = staff/principal.

Performance vs. Scalability

What It Is

Performance measures how fast a single request executes. **Scalability** measures how capacity grows when resources increase. A system can be performant but not scalable (fast at 1K requests but collapses at 10K), or scalable but not performant (handles 1M requests but each takes 5 seconds).

The critical mental model: Think of a highway. A sports car is performant (fast single trip) but a multi-lane freeway is scalable (moves more total cars per hour regardless of individual speed).

Core Dimensions

Performance — Single-Request Speed

Definition: How quickly one unit of work completes on a single node.

Characteristics:

- **Latency distribution (mean, p95, p99, p99.9)**

- **Cause:** The Microservice Fan-Out Multiplier — $P = (1 - 0.99)^N$. With 10 downstream services each at 99% reliability, the p99 experience is only 90.4%. Tail latency compounds multiplicatively across service hops.
- **Approach:**
 - Hedged requests — send speculative retries to replicas if first response exceeds latency threshold
 - Back pressure / load shedding — downstream returns 429, upstream slows down
 - Least-connections routing — avoid overloading already-slow instances
- **Trade-off:** Hedged requests increase total system load by 5–10% but improve p99 latency by 2–5x at the 99.9th percentile
- **Hedged Requests Deep Dive — When and How:**

```
Standard request pattern:
t=0ms: Send request to replica A
t=50ms: Still waiting → user sees 50ms latency

Hedged request pattern:
t=0ms: Send request to replica A
t=10ms: No response yet → send hedged request to replica B
t=12ms: Replica B responds → cancel A, return result
Result: 12ms latency instead of 50ms at p99
```

When to use hedged requests:

-  Read-heavy workloads (safe to duplicate)
-  High variance in response time ($p99/p50 > 3\times$)
-  Fan-out scenarios (search aggregation, parallel DB queries)
-  Write operations (creates duplicate writes)
-  Expensive operations (doubles compute cost for marginal gain) **Implementation pattern:**

```
def hedged_request(replicas, hedge_after_ms=10):
    tasks = []
    for i, replica in enumerate(replicas):
        delay = 0 if i == 0 else hedge_after_ms
```

```
tasks.append(delayed_call(replica, delay))
return first_completed(tasks) # cancel others
```

- **Resource saturation curves (CPU, memory, I/O)**

- **Cause:** Every resource has a utilization point beyond which latency spikes non-linearly (typically CPU > 70%, memory > 80%, I/O queue > 1)
- **Approach:** Auto-scaling triggers, circuit breakers to shed load, capacity planning to stay below saturation threshold
- **Trade-off:** Over-provisioning for headroom (2x peak) vs. scaling cost

- **Single-thread efficiency**

- **Cause:** Critical sections (locks, serialized processing) create sequential bottlenecks that limit throughput regardless of core count
- **Approach:** Profile hot paths with `perf`, minimize lock duration, use lock-free data structures
- **Trade-off:** Lock-free algorithms add complexity and are harder to verify correct
- **Lock vs Lock-Free Comparison — When to Use Each:**

Approach	Latency (uncontended)	Latency (contended)	Complexity	When to Use
Mutex	~25ns	1-10µs (context switch)	Low	Low contention (<10 threads), complex critical sections
Spinlock	~10ns	Burns CPU	Low	Very short critical sections (<100ns), real-time systems
RW Lock	~50ns read, ~100ns write	Readers don't block each other	Medium	Read-heavy (90%+ reads)
CAS (Compare-Exchange)	~10ns	~50ns (retry loop)	High	Counters, simple state

Approach	Latency (uncontended)	Latency (contended)	Complexity	When to Use
and-Swap)				machines, high contention
Lock-Free Queue	~20-50ns	~100ns	Very High	Producer-consumer, high throughput required

Real-world example — Metrics Counter:

```
// Mutex approach (simple, slower)
synchronized void increment() { counter++; }

// Atomic approach (faster, lock-free)
AtomicLong counter;
counter.incrementAndGet(); // CAS loop internally

// Performance: AtomicLong is 5-10x faster under contention
```

When NOT to use lock-free:

- Complex invariants spanning multiple variables
 - Already proven that locks aren't the bottleneck
 - Team unfamiliar with memory ordering semantics
- **Cache hit ratios & memory locality**
 - **Cause:** Poor data locality causes cache misses (L1 hit ~1ns, DRAM hit ~100ns — 100x penalty). Linked list traversal is a classic example.
 - **Approach:** Use contiguous arrays over linked structures, align data to cache lines (64 bytes), profile cache misses with `perf stat -e cache-misses`
 - **Trade-off:** Array-based structures have $O(n)$ insertion cost vs. $O(1)$ for linked lists
 - **Cache Level Comparison — When to Optimize for Each:**

Cache Level	Latency	Size	When to Optimize
L1	~1ns	32-64 KB	Hot loop data (loop counters, tight data structures)

Cache Level	Latency	Size	When to Optimize
L2	~3-5ns	256-512 KB	Frequently accessed per-core data
L3	~10-20ns	8-32 MB	Shared read-heavy data across cores
DRAM	~100ns	GBs	Everything else; optimize to keep working set in L3

- **Use case:** Image processing hot loop — keep pixel buffer in L1 by processing in 32 KB chunks
- **Use case:** Database query executor — keep index nodes in L3 by designing B-tree nodes = 1 cache line (64 bytes)
- **When NOT to optimize:** If working set > L3 size, focus on prefetching/batching over cache alignment

Scalability — Capacity Growth with Resources

Definition: How much additional capacity is gained per added resource unit.

Characteristics:

- **Horizontal scaling efficiency**
 - **Cause:** Amdahl's Law imposes a hard ceiling — if 10% of work is sequential, 10× speedup is the absolute maximum regardless of cores added
 - **Approach:** Minimize the sequential fraction (S); parallelize hot paths; use map-reduce patterns
 - **Trade-off:** Reducing S requires code restructuring — diminishing effort returns below $S < 5\%$
- **Coordination overhead growth**
 - **Cause:** Inter-node communication grows as $O(n)$, $O(n \log n)$, or $O(n^2)$ depending on topology. Full mesh = $O(n^2)$ heartbeats
 - **Approach:** Hierarchical clustering (cells, regions), gossip protocols ($O(\log n)$ messages), partition to reduce coordination set size
 - **Trade-off:** Hierarchical clustering adds latency for cross-cell coordination
- **Data partitioning & hotspot detection**
 - **Cause:** Uneven data distribution creates hotspots. 1% of keys can receive 90% of traffic (Zipf distribution)

- **Approach:** Consistent hashing with virtual nodes, rebalancing triggers, key-level replication for hot keys
- **Trade-off:** More partitions improve distribution but increase rebalancing complexity

Supporting Math — Amdahl's Law

Amdahl's Law gives the theoretical max speedup from parallelization:

$$\text{Speedup} = 1 / (S + (1 - S) / N)$$

Where:

S = fraction of work that is sequential (cannot be parallelized)

N = number of parallel workers

Worked example — Search indexing pipeline (S = 10% sequential for merging):

```
2 workers:  1 / (0.10 + 0.90/2) = 1.82x speedup
8 workers:  1 / (0.10 + 0.90/8) = 4.71x speedup
32 workers: 1 / (0.10 + 0.90/32) = 7.80x speedup
∞ workers:  1 / (0.10 + 0)      = 10x max — ever
```

Cause: Sequential fraction S is inherent to the problem (merge phase of map-reduce, global lock, DB write-ahead log serialization). **Approach:** Restructure to reduce S — not just throw more hardware.

Connects to 1.3: NUMA topology and cache coherence overhead add to effective S even in “parallel” code.

Real-World Comparison: Redis vs. Cassandra

Property	Redis (single-threaded core)	Cassandra (multi-threaded)
Single-request latency	<1ms (in-memory)	2–10ms
Throughput ceiling	~1M ops/s per node (CPU-bound)	~100K ops/s per node (I/O-bound)
Horizontal scale	Cluster sharding required	Native partitioning
Bottleneck	Single thread becomes hotspot	Lock contention at high concurrency

Takeaway: Redis wins on latency; Cassandra wins on horizontal write throughput for large datasets.

Trade-Off Matrix

Metric	Performance Focus	Scalability Focus
Optimization	Low-latency single request	High-throughput system growth
Bottleneck	Critical section optimization	Coordination overhead reduction
Failure mode	Resource exhaustion	Cascade through system boundaries
Testing	Load testing on single node	Stress testing across cluster
Example	Shaving 5ms off a DB query	Adding a new shard to handle 2x users

Cross-Concept Connections

Connection	Direction	Why
Amdahl's Law → Parallelism (Concurrency section)	Within 1.1	Both govern how many cores you can effectively use
Amdahl's Law → NUMA (1.3)	Cross-file	Cross-socket memory access adds effective S (coherence overhead)
Latency distribution → Latency Waterfall (Latency section)	Within 1.1	Latency waterfall breaks down the components that form the distribution
Scalability → Capacity Estimation	Within 1.1	Estimation determines how many nodes to provision for scale targets

Failure Modes

- **Thundering herd on cache miss**

- **Cause:** A cache key expires, and hundreds of simultaneous requests miss and hammer the DB simultaneously
- **Approach:**
 - Request coalescing — one request fetches, others wait
 - Stale-while-revalidate — serve stale data briefly while refreshing
 - Probabilistic early expiration — proactively refresh before TTL expiry
- **Code pattern (stale-while-revalidate):**

```
def get_with_coalescing(key):  
    val = cache.get(key)  
    if val is None:  
        with lock(key):  
            val = cache.get(key)      # only one fetcher proceeds  
            # re-check after lock  
            if val is None:  
                val = db.fetch(key)  
                cache.set(key, val, ttl=300)  
    return val
```

- **Trade-off:** Coalescing adds ~1ms per miss but prevents N:1 DB hammer; stale-while-revalidate trades consistency for latency
- **Metastable failure (retry death spiral)**
 - **Cause:** System appears healthy under normal load but enters a death spiral when slightly overloaded — retries pile on a slow DB, making it slower, causing more timeouts and more retries
 - **Approach:** Exponential backoff with jitter (not linear), retry budgets (max % of requests that can be retries), circuit breakers
 - **Trade-off:** Aggressive backoff increases latency for the retried request; liberal budgets risk cascade
- **Resource fragmentation**
 - **Cause:** Unused memory scattered across instances causes OOMs even when aggregate free memory appears sufficient. JVM heap fragmentation is a classic example.
 - **Approach:** `jemalloc` / `tcmalloc`, memory pools, object reuse
 - **Trade-off:** Memory pools add code complexity but reduce fragmentation at scale

Production Edge Cases

- **The 80/20 read skew:** Most OLTP systems are ~80% reads. Cache 20% of the data that satisfies 80% of reads (Zipf distribution). **Cause:** Natural power-law distribution of data popularity. **Approach:** Size caches based on working set, not total dataset.
- **Peak-to-average ratio:** Black Friday is 10x normal for e-commerce; viral tweets 100x normal for social. **Cause:** Human behavior patterns. **Approach:** Auto-scaling with predictive models, spot instance burst capacity.

Latency vs. Throughput

What It Is

Latency measures time per operation (how long one user waits). **Throughput** measures operations per time unit (how many users you can serve). These are fundamentally in tension: batching improves throughput by amortizing fixed overhead, but forces each individual item to wait longer for the batch to fill.

Core Dimensions

Latency — Time per Operation

Definition: End-to-end time from request submission to response receipt.

Characteristics:

- **Latency Waterfall components**
 - **Cause:** Total latency = Network RTT + Queue wait + Processing + DB query + Serialization + GC pause + Response serialization. Each component has a different magnitude and optimization strategy.
 - **Approach:** Profile which component dominates (distributed tracing with Jaeger/Zipkin, `perf` for CPU), optimize the bottleneck first (Amdahl's Law applies to latency too — fixing a 5ms component when the bottleneck is 50ms elsewhere yields 0 benefit)
 - **Typical breakdown:**

Total Latency =	
+ Network RTT (client ↔ LB)	~1-5 ms
+ Queue wait (LB → app)	~0-10 ms
+ Processing time	~1-50 ms

+ DB query (index hit)	~1-5 ms
+ DB query (disk seek)	~5-50 ms
+ Serialization overhead	~0.1-2 ms
+ GC pause time	~0-500 ms
+ Response serialization	~0.1-5 ms
Typical user-facing total:	~10-100 ms

- **Trade-off:** Reducing any component costs effort — always optimize the biggest contributor first. **Connects to 1.3:** DRAM hit = 100ns, NVMe = 100µs — these numbers inform the waterfall.
- **GC pause — the silent latency killer**
 - **Cause:** Java's Stop-The-World GC can pause 50–500ms. At p99.9, every user hits it periodically because GC runs on the critical path of all threads.
 - **Approach:** Tune heap size (avoid frequent full GCs), use G1GC/ZGC/Shenandoah (sub-10ms pause targets), or switch to a GC-free runtime (Go, Rust) for latency-critical paths
 - **Trade-off:** ZGC adds ~15% CPU overhead for sub-millisecond pauses; Go's GC is concurrent but can still cause ~100µs pauses
 - **GC Algorithm Comparison — Choosing the Right Collector:**

GC Algorithm	Pause Time	Throughput	Heap Size Limit	When to Use
Serial GC	50-500ms	Highest	<2 GB	Single-core apps, batch jobs where latency doesn't matter
Parallel GC	50-500ms	Very High	<64 GB	Batch processing, throughput > latency
G1GC (default Java 9+)	10-200ms	High	<32 GB	General purpose, predictable pause targets
ZGC (Java 15+)	<1ms		TB+	Ultra-low latency, trading

GC Algorithm	Pause Time	Throughput	Heap Size Limit	When to Use
		Medium (15% overhead)		throughput for consistency
Shenandoah	<10ms	Medium-High	<200 GB	Low latency with better throughput than ZGC
Go GC	100µs-2ms	High	Any	Concurrent, low pause, but not deterministic
Rust (no GC)	0ms	Highest	N/A	Zero-cost abstraction, real-time systems

Decision matrix for interviews:

```

p99 latency requirement < 10ms?
├ Yes → ZGC/Shenandoah or consider Go/Rust
└ No → G1GC is fine

Heap size > 32 GB?
├ Yes → ZGC (designed for large heaps)
└ No → G1GC

Trading 15% CPU for consistent latency acceptable?
├ Yes → ZGC
└ No → Tune G1GC or use language without GC

```

Real production example:

- Twitter moved from Parallel GC to G1GC → p99.9 latency improved 3×
- Discord moved from Go to Rust for read-heavy services → eliminated GC pauses entirely, CPU usage dropped 30%

Throughput — Operations per Time Unit

Definition: Number of operations completed per second across the system.

Characteristics:

- **Parallelism degree**

- **Cause:** Throughput is limited by the number of concurrent workers (cores, threads, connections). More workers → more parallelism → higher throughput ceiling, but coordination overhead grows with more workers.
- **Approach:** Match worker count to available resources ($GOMAXPROCS = \text{cores}$, thread pool size = cores + I/O wait factor)
- **Trade-off:** Too many workers → context switching overhead dominates; too few → resources idle

- **Resource efficiency (cache, batching, compression)**

- **Cause:** Fixed overhead per operation (syscall, lock acquisition, I/O) limits throughput. Batching amortizes fixed cost across multiple operations.
- **Approach:** Batch writes (Kafka `linger.ms`), use compression (Snappy/Zstd), enable HTTP keep-alive to avoid connection setup per request
- **Trade-off:** Batching increases p50 latency but improves p99 throughput

- **Coordination overhead**

- **Cause:** Locks, consensus (Paxos/Raft), distributed transactions all add per-operation overhead that grows with system size
- **Approach:** Minimize distributed coordination (use local decisions where possible), choose coordination-free CRDTs for eventually consistent systems
- **Trade-off:** Avoiding coordination means accepting weaker consistency

The Batching Trade-Off Illustrated

Without batching (low latency, low throughput):

```
t=0ms: Request A → DB write → response at 5ms
t=2ms: Request B → DB write → response at 7ms
t=4ms: Request C → DB write → response at 9ms
DB writes: 3 individual writes
```

With batching (higher latency, higher throughput):

```
t=0ms: Request A arrives, waits
t=2ms: Request B arrives, waits
t=4ms: Request C → flush batch → 1 bulk write for A+B+C
Responses: A at 9ms, B at 7ms, C at 5ms (avg latency increases)
DB writes: 1 bulk write (3× more efficient)
```

Real example — Kafka producer batching: `linger.ms=5` holds messages for 5ms to accumulate a batch. Throughput increases 10–100×; per-message latency adds 5ms worst case. For analytics pipelines this is fine; for payment confirmations, it is not.

Batching Strategy Comparison — Choosing the Right Approach:

Strategy	Max Latency Added	Throughput Gain	When to Use
No batching	0ms	1× (baseline)	Real-time payments, user-facing writes, strong consistency required
Time-based (flush every N ms)	N ms	10-50×	Analytics, logs, metrics (Kafka <code>linger.ms=5-50</code>)
Size-based (flush every N items)	Unbounded (waits for N items)	50-100×	High-throughput streams where latency doesn't matter
Hybrid (time OR size)	Max(time_limit)	20-80×	Most common: flush every 100 items OR 10ms, whichever first
Adaptive (adjust based on load)	Variable	Optimal	Complex: batch more under high load, less under low load

Decision framework:

```
User-facing synchronous operation?  
└ No batching (user waits for confirmation)  
  
Analytics/logging pipeline?  
└ Time-based (linger.ms=10-50)  
  
High-volume async writes (e.g., IoT sensor data)?  
└ Hybrid (size=1000 OR time=50ms)  
  
Variable load (bursty traffic)?  
└ Adaptive batching
```

Production examples: - **Kafka producer:** `batch.size=16KB, linger.ms=10` → waits 10ms or until 16KB accumulated - **Elasticsearch bulk API:** Batch 1000 documents OR 5MB OR 30s → optimizes index refresh - **DynamoDB BatchWriteItem:** Max 25 items per batch, application controls timing

Supporting Math — Little's Law

$$L = \lambda \times W$$

Where:

L = average number of requests in the system (queue + processing)

λ = arrival rate (requests per second)

W = average time a request spends in the system (latency)

Why it matters in interviews: If p99 latency is 200ms and the service has 50 concurrent threads, max throughput is:

$$\lambda = L / W = 50 / 0.200s = 250 \text{ RPS per instance}$$

Cause: This relationship is a law of stable systems — you cannot exceed it without adding threads or reducing latency. **Approach:** To serve 1,000 RPS: 4 instances. This gives a sizing formula interviewers love to see.

Trade-Off Matrix

Workload	Latency Priority	Throughput Priority	Example
User-facing API	p99 < 100ms critical	Secondary concern	Stripe checkout
Batch processing	Minutes acceptable	Maximize QPS	Nightly ETL jobs
Streaming	Jitter < 50ms	Sustained throughput	Live video
ML inference	<10ms for UX	Batching for GPU efficiency	Recommendation engine

Cross-Concept Connections

Connection	Direction	Why
Latency waterfall → DRAM/NVMe/HDD (1.3)	Cross-file	Hardware latency numbers fill the waterfall cells
Little's Law → Capacity Estimation	Within 1.1	Sizing formula: $\lambda = L/W$ directly

Connection	Direction	Why
		informs instance count
Batching trade-off → Kafka configuration	Within 1.1	Concrete production example of a throughput-vs-latency decision
GC pause → Go/Java async runtimes (1.2)	Cross-file	Runtime choice determines whether GC pauses exist

Failure Modes

- **Latency spikes from GC pause**
 - **Cause:** Stop-the-world GC freezes all threads for 50–500ms. At p99.9, every user hits it periodically
 - **Approach:** G1GC/ZGC/Shenandoah for low-pause GC; or GC-free runtimes (Go, Rust)
 - **Trade-off:** Low-pause GC adds CPU overhead; GC-free languages have steeper learning curves
- **Throughput collapse from lock contention**
 - **Cause:** A single contended lock serializes all threads, dropping throughput to single-core levels
 - **Approach:** Lock striping, read-write locks, lock-free data structures
 - **Trade-off:** All approaches increase code complexity

Interview Scenario ★★

Design Twitter’s home timeline with strict p99.9 latency requirements. How do you balance fan-out-on-write (better read latency) vs fan-out-on-read (better write throughput)?

Solution Framework

Tests dimension: Latency vs. throughput trade-off at system design scale.

Fan-out-on-write (Twitter’s historical approach): - On tweet creation, push tweet ID to every follower’s timeline list in Redis - Read latency: $O(1)$ — single Redis list read - Write latency:

O(followers) — Katy Perry (150M followers) causes 150M Redis writes per tweet - Production fix: Hybrid — fan-out-on-write for normal users (<100K followers), fan-out-on-read for celebrities

Fan-out-on-read (used for celebrities): - On timeline load, query accounts being followed → fetch their latest tweets - Read latency: $O(\text{followed} \times \text{parallelism})$ — parallel fan-in from ~100 accounts typically acceptable - Write latency: $O(1)$ - Cache effectiveness: Cache celebrity tweets separately (high reuse); cache timelines for active users only

Decision framework:

```
If user.follower_count < 100_000:
    use fan-out-on-write # write amplification is manageable
else:
    use fan-out-on-read # avoid 150M async writes per tweet
```

Key insight: There is no universally correct answer — interviewers want to see you reason through the trade-offs and identify the threshold where you'd switch strategies.

Capacity Estimation Techniques

What It Is

Back-of-the-envelope calculations inform every architectural decision. Underestimate and you'll fail under load. Overestimate and you waste millions in over-provisioning. Capacity estimation connects the abstract primitives (latency, throughput) to concrete infrastructure decisions (instance count, storage volume, network bandwidth).

Core Dimensions

Storage Estimation

Definition: How much disk/object storage is needed given user behavior and retention policy.

Characteristics:

- **Powers of Two (memory sizing intuition)**
 - **Cause:** Estimates require converting between orders of magnitude intuitively — knowing that $2^{30} = 1 \text{ GB}$ lets you compute storage needs without a calculator
 - **Approach:** Memorize the powers-of-two table:

$$2^{10} = 1 \text{ KB}$$

$$2^{20} = 1 \text{ MB}$$

$$2^{30} = 1 \text{ GB}$$

$$2^{40} = 1 \text{ TB}$$

- **Time shortcuts:** 1 day $\approx$ 100K seconds, 1 month $\approx$ 2.5M seconds, 1 year $\approx$ 30M seconds
- **Trade-off:** 10% error on time shortcuts is acceptable for estimation (not budgeting)

- **Storage Derivation Chain**

- **Cause:** Total storage depends on multiple factors: user count, actions/day, data size per action, replication factor, retention period
- **Approach:** Use the chain formula:

```
Users × Actions/day × Size/action = Daily storage
Daily storage × Replication factor × Retention days = Total storage
Total storage ÷ Storage node capacity = Min nodes
Min nodes × (1 + headroom%) = Provisioned nodes
```

- **Trade-off:** Conservative headroom (2×) vs. aggressive (1.2×) — depends on product maturity (startup vs. stable)

Throughput Estimation

Definition: Peak requests per second the system must handle.

Characteristics:

- **Little's Law for instance count**

- **Cause:** Given target throughput λ and per-instance latency W , you need L/λ instances where L = concurrent capacity per instance
- **Approach:** $\lambda = L / W \rightarrow \text{instances} = \text{target_RPS} / (\text{concurrent_threads} \times (1000\text{ms} / \text{latency_ms}))$
- **Trade-off:** More instances = higher cost but lower p99 latency

- **Peak-to-average ratio**

- **Cause:** Real-world traffic is not uniform. Black Friday = 10× normal; virality = 100× normal for social platforms. Provisioning for average means failure at peak.
- **Approach:** Size for 2× peak (stable products) or 5× peak (new/viral products); use auto-scaling and spot instances for burst capacity
- **Trade-off:** Provisioning for peak = over-provisioned 99% of the time; auto-scaling may lag behind a sudden spike

- **Hot key problem (Zipf distribution)**

- **Cause:** 1% of data drives 90% of requests. A viral tweet from a celebrity causes one Redis key to receive millions of reads/second.
- **Approach:** CDN for top 1% of content, key-level replication (distribute hot key across multiple nodes), local caching (client-side cache for hot content)
- **Trade-off:** CDN adds caching complexity; key-level replication creates write amplification

Worked Example: Instagram Photo Storage

Assumptions:

- 1 billion users, 10% active daily (100M DAU)
- Average 2 photos uploaded per active user per day
- Average photo size: 2 MB (post-compression)
- Replication factor: 3 (HA across AZs)
- Retention: indefinite

Daily new storage:

$$100\text{M users} \times 2 \text{ photos} \times 2 \text{ MB} = 400 \text{ TB/day}$$

Total storage after 5 years:

$$400 \text{ TB/day} \times 365 \times 5 \times 3 \text{ replicas} = \sim 2.2 \text{ EB}$$

Storage nodes (10 TB per node):

$$2.2 \text{ EB} / 10 \text{ TB} = 220,000 \text{ nodes} \times 1.3 \text{ headroom} = \sim 286,000 \text{ nodes}$$

Bandwidth (peak 2× average):

$$400 \text{ TB/day} / 86,400\text{s} \times 2 \text{ peak} = \sim 9 \text{ Gbps ingress}$$

Interview signal: Know that Instagram uses S3-compatible object storage (not raw nodes) and deduplication reduces footprint by ~30%.

Worked Example: WhatsApp Message Throughput

Assumptions:

- 2 billion users, 50% send 40 messages/day
- Average message: 100 bytes (text), 10% include media (50 KB avg)
- Messages stored 30 days (then archived)

Message throughput:

$$1\text{B users} \times 40 \text{ msgs} / 86,400\text{s} = \sim 460\text{K messages/second}$$

Storage for text messages (30 days):

$$1\text{B} \times 40 \times 100 \text{ B} \times 30 = 120 \text{ TB/month}$$

Storage for media (10% of messages):

$$1\text{B} \times 4 \times 50 \text{ KB} \times 30 = 6 \text{ PB/month}$$

Bandwidth:

460K msg/s × 100B = ~46 MB/s (text, negligible)
46K media/s × 50 KB = ~2.3 GB/s (media upload — this is the constraint)

Trade-Off Matrix

Factor	Conservative Estimate	Aggressive Estimate	When to Use Each
Headroom	2x over expected peak	1.2x over expected	New product vs. stable
Data model	Worst-case row size	Average + compression	Unknown vs. measured
Network	Cross-AZ bandwidth cost	Same-AZ assumptions	Multi-region vs. single
Growth	3x annual	1.2x annual	Startup vs. mature

Cross-Concept Connections

Connection	Direction	Why
Storage estimates → HDD/SSD costs (1.3)	Cross-file	Cost/TB from 1.3 feeds storage cost calculations
Throughput estimates → Little's Law	Within 1.1	Little's Law provides the instance-counting formula
Hot key problem → CDN/latency	Within 1.1	Hot keys degrade latency; CDN mitigates both

Concurrency vs. Parallelism

What It Is

Concurrency structures multiple tasks to make progress — they may not execute simultaneously (e.g., single-threaded event loop switching between tasks). **Parallelism** executes tasks literally simultaneously on multiple CPUs.

Concurrency (time-sliced):

CPU: —A—A—B—A—B—B—A—

(A and B interleave, only one runs at a time)

Parallelism (simultaneous):

CPU 1: —A—A—A—A—

CPU 2: —B—B—B—B—

(A and B run at the same time on different cores)

The key insight: You can have concurrency without parallelism (Node.js on a single core), parallelism without useful concurrency (two threads both blocked waiting for the same lock), or both (Go goroutines on a multi-core machine).

Core Dimensions

Concurrency Models

Definition: How tasks are structured to interleave execution, regardless of parallelism.

Characteristics:

- **Event Loop (Node.js, Nginx)**

- **Cause:** I/O-bound workloads spend 95% of time waiting. A single thread handling many connections via epoll/kqueue avoids context switch overhead.
- **Approach:** Register FDs with epoll → event loop dispatches callbacks when I/O is ready → never block
- **Trade-off:** Any blocking call stalls ALL connections; CPU-bound work must be offloaded to worker threads or child processes. Best for high-concurrency I/O (proxies, chat).

- **Coroutines / async/await (Python, JS, Rust Tokio)**

- **Cause:** Cooperative yield points let a task voluntarily suspend at I/O wait, allowing other tasks to run on the same thread
- **Approach:** `await` keyword marks yield points; runtime schedules resumed coroutines when I/O completes
- **Trade-off:** Async frames produce poor stack traces; any blocking call stalls the entire runtime. Best for high-concurrency I/O services.

- **Actor Model (Erlang, Elixir, Akka)**

- **Cause:** Shared state causes race conditions. Actors isolate state entirely and communicate via message queues, eliminating races by design.
- **Approach:** Each actor has a mailbox; messages are processed one at a time. Actors can spawn child actors for fault isolation.

- **Trade-off:** Message overhead $\sim 5\mu\text{s}$ vs $\sim 100\text{ns}$ for shared memory. Best for distributed systems where fault tolerance matters more than raw speed.

Parallelism Models

Definition: How tasks execute simultaneously on multiple processors.

Characteristics:

- **Shared Memory (threads, locks)**
 - **Cause:** CPU-bound work benefits from multiple cores. Threads within the same process share heap, making data access fast ($\sim 100\text{ns}$ via shared memory) but requiring synchronization.
 - **Approach:** Thread pool sized to core count; lock striping to reduce contention; use read-write locks for read-heavy access patterns
 - **Trade-off:** Locks create contention, deadlock risk, and debugging difficulty. Best for computation on a single machine.
- **Distributed (message passing, MapReduce)**
 - **Cause:** Work that exceeds a single machine's capacity must be distributed across nodes
 - **Approach:** Partition data, assign work units to nodes, communicate results via network. MapReduce/Spark for batch; Kafka Streams/Flink for streaming.
 - **Trade-off:** Network latency ($\sim 500\mu\text{s}$ in-DC) vs. shared memory ($\sim 100\text{ns}$). Best for data-intensive workloads at scale.
- **SIMD / GPU**
 - **Cause:** Data-parallel operations (matrix multiplication, image processing) apply the same instruction to many data elements simultaneously
 - **Approach:** Use CUDA/SYCL for GPU; compiler auto-vectorization for SIMD; design data layout for coalesced memory access
 - **Trade-off:** Requires specialized hardware and programming model. Best for ML training, image processing, scientific computing.

Synchronization Primitives Compared

Mutex (exclusive lock):

- ✓ Simple correctness model
- ✗ Deadlock risk with multiple locks; priority inversion

Read-Write Lock:

- ✓ Concurrent reads; exclusive writes
- ✗ Write starvation if reads are frequent

✓ Semaphore (counting):

- ✗ Controls access to N resources (e.g., DB connection pool)
- Not tied to ownership; easy to misuse

✓ Atomic Compare-and-Swap (CAS):

- ✗ Lock-free; no context switch overhead
- ABA problem; retry loops under high contention

✓ Channel (Go-style):

- ✗ Communicating Sequential Processes; no shared state
- Buffering strategy must be chosen carefully

Trade-Off Matrix

Model	Synchronization Cost	Memory	Debugging	Throughput Ceiling
Shared Memory	High (locks, atomics)	Shared	Complex (race conditions)	CPU cores
Message Passing	None (isolated state)	Independent	Simpler	Network bandwidth
Event Loop	None (single-threaded)	Shared	Straightforward	Event loop latency
Actor	Low (mailbox)	Isolated per actor	Moderate	Message queue throughput

Cross-Concept Connections

Connection	Direction	Why
Concurrency models → Process vs Thread (1.2)	Cross-file	Event loops = single process; shared memory = multi-threaded; actors = multi-process
Parallelism → Amdahl's Law	Within 1.1	Amdahl's Law governs the speedup ceiling for all parallelism models
Locks → False sharing (1.3)	Cross-file	

Connection	Direction	Why
		Two variables on the same cache line cause 100x slowdown under contention
Synchronization → N-tier architecture (1.2)	Cross-file	Distributed locking patterns appear in multi-service coordination

Failure Modes

- **Race condition**

- **Cause:** Two goroutines increment a counter without synchronization — result is non-deterministic because the read-modify-write cycle interleaves unpredictably
- **Approach:** Use `sync/atomic`, mutex, or channel-based synchronization
- **Trade-off:** Synchronization reduces throughput; the right tool depends on contention level

- **Deadlock**

- **Cause:** Thread A holds L1, waits for L2. Thread B holds L2, waits for L1. Both block forever.
- **Approach:** Always acquire locks in the same global order; use lock timeout + retry; use lock hierarchy (assign numeric levels)
- **Trade-off:** Strict ordering requires discipline in code reviews; timeouts mask legitimate wait scenarios

- **ABA problem (CAS)**

- **Cause:** Thread reads value A, another thread changes A→B→A. Thread 1's CAS succeeds thinking nothing changed — but intermediate state was modified.
- **Approach:** Stamped references (version counter alongside pointer, e.g., Java `AtomicStampedReference`)
- **Trade-off:** Stamped references consume more memory per atomic variable

- **Livelock**

- **Cause:** Threads keep yielding to each other and making no progress — like two people in a hallway each stepping aside for the other, blocking indefinitely
- **Approach:** Random backoff (exponential + jitter) so threads fall out of sync
- **Trade-off:** Backoff adds latency for the yielding thread

Interview Question

Your service handles 10K concurrent WebSocket connections. Compare Node.js event loop, Go goroutine pool, and Java thread-per-connection approaches.

Solution Framework

Tests dimension: Concurrency model selection based on workload characteristics.

Node.js (event loop): - Memory: ~2 KB per connection - Max connections: 10K is trivial; 1M+ with tuning - Critical weakness: Any blocking call stalls ALL 10K connections - Best for: Chat servers, notification hubs

Go (goroutines): - Memory: ~2 KB initial stack per goroutine - Max goroutines: 1M+ practical; M:N scheduler - Blocking calls only suspend that goroutine - Best for: Mixed I/O + light CPU; production-proven at Cloudflare, Twitch

Java (thread-per-connection, pre-Loom): - Memory: ~512 KB–1 MB per thread - Max threads: ~10K practical limit - Java 21+ Virtual Threads ~1KB each, comparable to Go goroutines

Recommendation matrix:

10K connections, mostly idle:	Node.js or Go
10K connections, CPU-heavy work:	Go or Java thread pool
100K+ connections:	Go or Java 21 Virtual Threads

Additional Interview Question

Design a distributed job queue (like Sidekiq or Celery) that guarantees exactly-once execution for 100K jobs/second.

Solution Framework

Tests dimension: Concurrency + state management at scale.

Why “exactly-once” is hard: - Worker crashes after executing but before ACKing → duplicate - Network partition causes broker to re-deliver

Architecture:

1. **At-least-once delivery:** Redis or Kafka with consumer ACK
2. **Idempotency key to achieve exactly-once:**


```
def process_job(job):
    key = f"processed:{job.id}"
    if redis.set(key, "1", nx=True, ex=86400):
        execute(job)
```

3. **For non-idempotent operations:** Two-phase commit with intent log

4. **Scale to 100K jobs/s:** Kafka with 100 partitions → 100 consumers → 1K jobs/s each. DLQ for jobs that fail 3x.

Key insight: Exactly-once delivery is impossible at the transport layer; you achieve it via idempotent consumers at the application layer.

Stateful vs. Stateless Architecture

What It Is

Stateful services store session data locally in memory or on disk. **Stateless** services externalize all state to a shared store (Redis, database). Stateless enables horizontal scaling but introduces network hops and consistency challenges.

Core Dimensions

Stateful Architecture

Definition: Service instances hold session/state locally; clients must route to the same instance.

Characteristics:

- **Low-latency local access**
 - **Cause:** State is in local memory — 0 network hops, ~100ns DRAM access. No external dependency needed for read/write
 - **Approach:** Store session data in local hash map or embedded cache (EHCache, Caffeine)
 - **Trade-off:** Local state is fast but fragile — instance crash loses all sessions
- **Sticky session requirement**
 - **Cause:** Load balancer must route each client to the same instance consistently, otherwise the client's state is invisible
 - **Approach:** Cookie-based routing (JSESSIONID), IP hash, or application-level routing token
 - **Trade-off:** If the target instance restarts or is removed from pool, the session is lost regardless of routing

Stateless Architecture

Definition: All shared state is externalized; any instance can serve any request.

Characteristics:

- **Pure horizontal scaling**

- **Cause:** No local state means no routing constraints — add/remove instances freely based on load
- **Approach:** Externalize all session state to Redis, database, or object store; instances are interchangeable
- **Trade-off:** Adding +1 network hop to state store adds 0.5–5ms per request (Redis ~500µs, DB ~5ms)

- **JWT revocation problem**

- **Cause:** JWTs are self-contained — validation requires no DB lookup. But this means you can't revoke a token before its expiry (e.g., compromised account, user logout)
- **Approach** (in order of complexity):
 1. Short expiry (5–15 min) + refresh tokens — limits damage window
 2. Token blocklist in Redis — check on every request, adds ~1ms
 3. Token version in DB — increment on logout; token's `ver` claim must match DB
- **Trade-off:** Each mitigation adds more state to a “stateless” system, eroding the stateless advantage

Real-World Comparison: PHP Sessions vs. JWT

Property	PHP file-based sessions (stateful)	JWT tokens (stateless)
State location	Disk on the server that created session	Inside the token itself
Scaling	Sticky session or NFS share required	Any instance validates token
Invalidation	Easy (delete session file)	Hard (can't revoke without blocklist)
Payload size	Small (just session ID)	Larger (all claims embedded)
Example	Legacy monolith e-commerce	Modern SPA with API backend

Trade-Off Matrix

Aspect	Stateful	Stateless
Scaling	Requires sticky sessions	Pure horizontal scale
Failure	Session lost on crash	Session survives instance death
Latency	0 network hops to state	+1 network hop to state store
Consistency	Trivial (local)	Complex (distributed writes)
Cost	Low (no external store)	Higher (Redis cluster cost)

Cross-Concept Connections

Connection	Direction	Why
Stateful → Capacity Estimation	Within 1.1	Sticky sessions affect how you estimate node count (affinity constraints)
Stateless → HA Math	Within 1.1	State store availability determines overall system availability
State store → Redis (1.3 memory)	Cross-file	Choosing DRAM-hosted state store ties back to memory latency numbers
Stateless → N-tier architecture (1.2)	Cross-file	Stateless app tier with stateful DB tier is the 3-tier pattern

Production Edge Cases

- **Session affinity failure:** Load balancer maintains routing; if target instance restarts, session is lost. **Cause:** No state replication. **Approach:** Pair with session replication or external store. **Trade-off:** Replication writes cascade to every instance.
- **Thundering herd on state store:** If Redis goes down, all instances lose backing store simultaneously. **Cause:** Single shared state store = single point of failure. **Approach:** Design for Redis HA with Sentinel or Cluster mode. **Trade-off:** Sentinel adds 10s failover window.
- **State explosion:** Shopping cart sessions grow unboundedly. **Cause:** No TTL enforcement. **Approach:** Enforce TTL and max payload size on session data. **Trade-off:** Short TTIs log users out more frequently.

Service Level Metrics (SLA / SLO / SLI)

What It Is

- **SLI (Service Level Indicator):** The actual measured metric. “Our measured availability this month was 99.97%.”
- **SLO (Service Level Objective):** Internal target your team owns. Set *higher* than SLA to give a safety buffer. If SLA = 99.9%, set SLO = 99.95%.
- **SLA (Service Level Agreement):** Legal contract with customers — violating it means financial penalties (credits, refunds).

SLI (measured) → SLO (internal target) → SLA (customer contract)
99.97% 99.95% 99.90%
↑ buffer → ↑ buffer →

Core Dimensions

Availability Measurement (Nines)

Definition: The fraction of time the service is operational, expressed as a percentage.

Characteristics:

- **Availability nines table**
 - **Cause:** Each nine represents a 10x reduction in allowed downtime. Going from 99% to 99.9% costs exponentially more engineering effort.

- **Approach:** Pick the nine's target that matches your business needs — not every system needs 99.999%
- **Key data:**

Availability	Annual Downtime	Monthly Downtime	Use Case
99%	87.6 hours	7.3 hours	Internal tools
99.9%	8.76 hours	43.2 minutes	Standard SaaS
99.95%	4.38 hours	21.6 minutes	Business-critical
99.99%	52.6 minutes	4.3 minutes	Payment systems
99.999%	5.26 minutes	26 seconds	Telecom, life-critical

Error Budget

Definition: The acceptable amount of downtime/errors within the SLO period.

Characteristics:

- **Error budget calculation**

- **Cause:** Error budget = $(1 - \text{SLO}) \times \text{total requests}$. It quantifies how much unreliability is “acceptable” per period.
- **Approach:**

99.9% SLO on 1,000,000 requests/day → 1,000 errors allowed/day
 Burn rate = actual error rate / target error rate

- **Trade-off:** Tight error budgets force deployment discipline but can slow innovation

- **Burn rate alerting**

- **Cause:** Detecting SLO violations early requires alerting on burn rate, not raw error count. A fast burn (2% budget/hour) needs immediate response; a slow burn (10% budget/3 days) can wait for sprint planning.
- **Approach:**

Alert	Condition	Action
Fast burn page	2% of monthly budget in 1 hour	Page on-call

Alert	Condition	Action
Fast burn ticket	5% in 6 hours	Create P1 ticket
Slow burn	10% in 3 days	Sprint review

- **Trade-off:** More sensitive alerts catch issues faster but cause alert fatigue. Multi-window burn rate alerting balances sensitivity vs. noise.

Trade-Off Matrix

Alerting Strategy	Fast Burn	Slow Burn
Signal	Immediate paging	Warning in Slack
Response	Halt deployments	Reduce deploy velocity
False positive cost	Immediate interruption	Delayed reaction

Cross-Concept Connections

Connection	Direction	Why
SLOs → HA Mathematics	Within 1.1	HA design must achieve the SLO target; availability math proves the design
SLI measurement → Linux Diagnostics (1.2)	Cross-file	<code>iostat</code> , <code>vmstat</code> , APM tools provide SLI data
Error budget → Deploy frequency (1.2)	Cross-file	Error budget burn rate determines whether deploys are halted

Failure Modes

- **Alert fatigue**

- **Cause:** More than 3 pages per on-call shift causes engineers to ignore pages. Over-alerting is worse than under-alerting because it trains people to dismiss alarms.
- **Approach:** Multi-window burn rate alerting (Alertmanager), deduplicate related alerts into single incidents, tier alerts by severity with appropriate response SLAs
- **Trade-off:** Aggregating alerts may hide correlated failures that need individual investigation

- **SLO gaming**

- **Cause:** Teams optimize to the SLO metric, not actual user experience. Example: excluding slow health check requests from SLI calculation inflates apparent availability.
- **Approach:** Multiple SLOs (latency, error rate, correctness), user journey synthetic testing (measure real user paths, not internal health checks)
- **Trade-off:** More SLOs = more monitoring infrastructure, potential for analysis paralysis

Interview Question

Your SLO is 99.9% availability. A bug in a deploy causes 15% error rate for 20 minutes before rollback. How do you calculate budget impact, and what process changes do you recommend?

Solution Framework

Tests dimension: Error budget mechanics and incident response improvement.

Budget calculation:

Monthly error budget at 99.9% = 43.2 minutes total downtime
Incident: 15% error rate for 20 minutes

Effective downtime = 20 min × 15% error rate = 3 minutes burned
Remaining budget = 43.2 - 3 = 40.2 minutes this month

Process analysis: - 20 minutes to detect + rollback is too long. Target: <5 minutes - 20 minutes to rollback → deploy pipeline is too slow

Recommendations: 1. **Canary deployment:** Route 1% of traffic; auto-rollback if error rate spikes 2.

Feature flags: Decouple deploy from release; rollback in <30s 3. **Automated rollback:** p99 latency > 2× baseline for 2 consecutive minutes → auto-rollback 4. **Alerting:** Current burn rate = 6.9% in 20 min → should have fired fast-burn alert at ~2% (4 min in)

High-Availability Mathematics

What It Is

Designing for availability means calculating failure domains and their combined effect on system reliability. The math tells you exactly where adding redundancy has diminishing returns.

Core Dimensions

Serial Availability (Chain)

Definition: Components arranged in series, all must work — system availability is the product of all component availabilities.

Characteristics:

- **Serial compounding effect**
 - **Cause:** Every component must be available for the system to serve a request. A single point of failure anywhere in the chain takes the whole system down.
 - **Approach:** Reduce the number of components in the critical path; increase each component's individual availability
 - **Math:** $\text{System} = A_1 \times A_2 \times A_3 \times \dots \times A_n$
 - **Example:** LB (99.9%) × App (99.95%) × DB (99.99%) = 99.84%. A 3-tier system with all components at 99.9% gives only 99.7% — worse than any single component.
 - **Trade-off:** Each redundant component added to the chain for reliability actually decreases serial availability. Real fix: make components redundant in parallel, not additive in series.

Parallel Availability (Redundancy)

Definition: Components in parallel — any one working is sufficient.

Characteristics:

- **Parallel availability amplification**
 - **Cause:** If one replica can fail, two replicas mean both must fail simultaneously for the system to be down — failure probability goes from A to A^n .
 - **Approach:** Add redundant replicas for critical components; use N+2 for truly critical systems
 - **Math:** $\text{System} = 1 - (1 - A_1) \times (1 - A_2) \times \dots \times (1 - A_n)$
 - **Example:** 3 DB replicas each at 99.9%: $1 - (0.001)^3 = 99.9999999\%$
 - **Trade-off:** Each additional replica adds cost with diminishing returns (see below)

- **Diminishing returns cliff**

- **Cause:** The parallel formula is exponential — each additional replica multiplies an already-small failure probability, yielding tiny marginal gains
- **Approach:** Standard is 3 replicas — you get 99.9999% from 99.9% components. Going from 3 to 4 recovers only 28 seconds/year.
- **Data:**

Replicas	Availability	Annual Downtime	Marginal Gain
1	99.900%	8.76 hours	—
2	99.999%	5.26 minutes	+8.7 hours recovered
3	99.9999%	31.5 seconds	+5 minutes recovered
4	99.99999%	3.15 seconds	+28 seconds recovered

- This is why “3 replicas” is the standard across databases (RDS, Cassandra), Kafka topic replication, and HDFS.

Real-World Architecture Analysis

AWS Multi-AZ RDS (3 AZs):

Primary (AZ-1): 99.95% × Replica (AZ-2): 99.95% × Replica (AZ-3): 99.95%
Parallel: $1 - (0.0005)^3 = 99.999999875\%$ theoretical

Reality: Failover takes 30-60s, so measured availability $\approx 99.95\%$

Cause: Hardware redundancy is necessary but not sufficient. **Approach:** Failover *speed* determines observed availability, not component count alone.

Trade-Off Matrix

Strategy	Availability Gain	Cost	Complexity	Example
Active-Passive HA	High (removes SPOF)	2x resource cost	Low	RDS Multi-AZ
Active-Active	Very high (no failover time)	2x resource cost	High	Global Cassandra
Degraded-mode operation	Medium (keeps serving)	Engineering cost	Medium	Read-only mode during outage

Strategy	Availability Gain	Cost	Complexity	Example
Circuit breaker	Medium (prevents cascade)	Low	Low	Hystrix / Resilience4j

Cross-Concept Connections

Connection	Direction	Why
HA Math → SLO Targets	Within 1.1	HA math proves you can meet the SLO. If your target is 99.99%, the math must support it.
Serial availability → N-tier architecture (1.2)	Cross-file	Each tier boundary is a serial availability component
Parallel replicas → SSD/HDD failure rates (1.3)	Cross-file	Disk MTBF feeds into replica count decisions
Circuit breaker → N-tier cascade failures (1.2)	Cross-file	Circuit breakers prevent tier failure propagation

Failure Modes

• Split-brain

- **Cause:** Network partition causes both replicas to believe they are primary and accept writes. Data diverges — reconciliation may be impossible.
- **Approach:** Require quorum writes (majority of replicas must acknowledge) — sacrifices availability for consistency (CAP theorem). Raft uses leader lease with randomized timeouts.
- **Trade-off:** Quorum reduces write availability during partitions. With 3 replicas, you need 2/3 — one node failure + partition = no writes possible.

- **Byzantine failures**

- **Cause:** A replica that is “up” but returning corrupted data is worse than a failed replica. Traditional replication assumes crash-stop failures; Byzantine behavior (arbitrary faults) requires additional safeguards.
- **Approach:** Consensus protocols (Raft, Paxos) with checksummed data; require $\geq 2f+1$ replicas to tolerate f Byzantine faults
- **Trade-off:** Byzantine fault tolerance requires 2x more replicas and 3x more messages per write

- **Clock drift**

- **Cause:** Time-based failover (heartbeat timeout) triggers incorrectly if NTP diverges. A fast-running clock on one node can cause premature timeouts; a slow-running clock can delay failover detection.
- **Approach:** Use monotonic clocks for elapsed-time checks (not wall clocks). Raft uses randomized timeouts (150–300ms) to avoid split votes from synchronized clock drift.
- **Trade-off:** Randomized timeouts increase failover latency variance

Interview Question

Design a distributed rate limiter for a global API serving 1M RPS with p99 latency under 20ms across 3 regions. What is your availability target, and how do you achieve it?

Solution Framework

Tests dimension: HA mathematics applied to a real system with tight latency constraints.

Requirements Analysis:

```
1M RPS = ~86B requests/day
p99 < 20ms → must be in-memory (no cross-region calls on hot path)
3 regions → consistency vs. availability trade-off required
```

Architecture Options:

Option 1 — Local rate limiting (per-region): - Each region enforces limit/3 independently - Latency: ~1ms (local Redis read) - Weakness: User can exceed global limit by up to 3x - When to choose: Approximate limiting acceptable (DoS prevention)

Option 2 — Centralized Redis (one region): - Cross-region RTT = 80–200ms — **fails the 20ms requirement** - Never correct for this use case

Option 3 — Token Bucket per Region + Async Global Sync  :

Per request:

1. Check local token bucket (Redis, <1ms)
2. Allow or deny based on local state

Background (every 1s):

1. Report local usage to central counter
2. Receive global adjustment
3. Update local token bucket ceiling

Result: p99 < 5ms on hot path

Availability Target: 99.99% — active-active with failover: - Each region's Redis is Cluster mode (3 masters × 3 replicas) - Fail open on Redis failure (allow all requests) rather than fail closed - Circuit breaker: bypass rate limiter if Redis errors > 1% for 10s

Key insights: 1. Perfect rate limiting ≠ 100% availability. Trade minor over-allocation for system resilience. 2. The 20ms p99 eliminates any synchronous cross-region call — design accordingly. 3. “Fail open” is almost always right for rate limiters (unlike auth where you fail closed).

Quick Reference: Interview Signals Cheat Sheet

When interviewer asks...	Strong signal to show
“How would you scale this?”	Identify the bottleneck first (CPU / I/O / memory / network), then prescribe horizontal vs. vertical
“What’s your availability target?”	Quote nines with downtime budget; link to SLO/SLA distinction
“How do you handle cache misses?”	Request coalescing + stale-while-revalidate; estimate thundering herd impact
“How many servers do you need?”	Use Little’s Law: $L = \lambda W$; then add headroom for peak burst
“What happens if component X fails?”	Walk the failure domain; describe detection, mitigation, and recovery steps
“How do you test this?”	Unit → integration → load test (single node) → chaos/stress test (cluster)